Preparación del entorno para el proyecto S-MIPS

Arquitectura de Computadoras · Curso 2026-2027
Guía complementaria de instalación y comprobación · 30 de octubre de 2026

Esta guía explica cómo preparar la computadora para el proyecto publicado S-MIPS 2026-2027. La especificación es `docs/guide.pdf` de ese repositorio; esta guía la complementa y no modifica sus requisitos ni sus criterios de evaluación.

VHDL es opcional y tiene límites concretos: solo pueden implementarse en VHDL, como máximo, dos de estos tres componentes: `Register_File`, `ALU` e `Instruction_Decoder`. El resto de la CPU debe construirse con circuitos gráficos. Si no vas a utilizar VHDL, basta con completar las secciones 1-3; no necesitas instalar Questa. Las condiciones completas se resumen en la sección 7.

1. Qué necesitas

Herramienta	Para qué se utiliza	Cuándo hace falta
Logisim Evolution **5.0.0**	Abrir la plantilla, construir y simular circuitos	Siempre
Java/JDK **21 o posterior**	Ejecutar el JAR, el lanzador y las herramientas del proyecto	Los instaladores de Logisim incluyen Java para la aplicación, pero el lanzador y las herramientas necesitan Java disponible en la terminal; para ejecutar los fuentes `.java`, instala un JDK
MARS **4.5**	Ensamblar programas y obtener la salida de referencia	Para el flujo de ensamblado y pruebas de la orientación; no para abrir el circuito ni cargar una imagen ya preparada
Questa FPGA **Starter Edition** y su licencia gratuita	Compilar y simular componentes VHDL dentro de Logisim	Solo si utilizas VHDL

Las instrucciones principales son para computadoras Intel/AMD de 64 bits. No confundas `amd64`/`x86_64` con `aarch64`/ARM. Que Logisim tenga instalador para ARM no garantiza que Questa pueda utilizarse en esa computadora.

Usa **5.0.0**, la versión con la que se preparó la plantilla, aunque aparezcan versiones posteriores. Descarga desde la publicación oficial de Logisim Evolution 5.0.0, no desde una página que ofrezca el Logisim antiguo.

2. Instalar y abrir Logisim Evolution

Windows: instalación habitual

1. En la publicación oficial, despliega **Assets**.
2. Descarga `logisim-evolution-5.0.0-amd64.msi`.
3. Ejecuta el instalador y completa el asistente.
4. Abre **Logisim Evolution** desde el menú Inicio.
5. En **Ayuda → Acerca de…**, verifica que indique **5.0.0**.

Este instalador incluye Java. No necesitas instalar Java por separado para abrir la aplicación desde su acceso directo; esto no garantiza que `java` esté disponible en la terminal para las herramientas del proyecto. También existe `logisim-evolution-5.0.0-windows-amd64.zip`: extráelo completo antes de ejecutar el programa; no ejecutes archivos desde dentro del ZIP. Los nombres exactos están en los archivos oficiales de esta versión.

Linux: Ubuntu, Debian y derivados

1. Descarga `logisim-evolution_5.0.0_amd64.deb` desde la misma publicación.
2. Abre una terminal en la carpeta donde lo descargaste.
3. Instálalo con:

```
sudo apt install ./logisim-evolution_5.0.0_amd64.deb
```

4. Abre la aplicación desde el menú de aplicaciones.
5. Comprueba **Ayuda → Acerca de… → 5.0.0**.

El paquete incluye Java. Para Fedora y otras distribuciones con RPM existe `logisim-evolution-5.0.0-1.x86_64.rpm`. Si tu distribución no utiliza esos formatos, puedes elegir el JAR.

No desinstales versiones anteriores sin necesidad, pero evita confundir sus accesos directos con el de 5.0.0. Un comando o un doble clic en un `.circ` podría seguir abriendo una instalación vieja. Comprueba siempre la versión de la ventana que realmente se abrió.

Alternativa portable: JAR en Linux o Windows

Esta alternativa permite elegir explícitamente qué versión se abre y resulta útil para diagnosticar problemas.

1. Instala un **JDK 21 o posterior**; por ejemplo, Eclipse Temurin. El JDK permite abrir el JAR y ejecutar los fuentes Java que utilizan las herramientas entregadas.
2. Descarga `logisim-evolution-5.0.0-all.jar` de la publicación oficial.
3. Abre una terminal en la carpeta del JAR. En Windows utiliza **PowerShell**.
4. Comprueba y abre:

```
java -version
java -jar logisim-evolution-5.0.0-all.jar
```

La primera orden debe mostrar una versión de Java igual o superior a 21. Si Java no se reconoce, revisa su instalación y abre una terminal nueva. Estos requisitos y la inclusión de Java en los paquetes están documentados en el README oficial de 5.0.0.

Si los textos se superponen por el escalado de pantalla, cierra esa ventana y prueba:

```
java -Dflatlaf.uiScale=1 -jar logisim-evolution-5.0.0-all.jar
```

Después aumenta el tamaño del circuito con el zoom de Logisim. No modifiques la plantilla para compensar una versión incorrecta o un problema de escalado.

MARS y las rutas de las herramientas

La orientación utiliza **MARS 4.5** para ensamblar y comparar resultados. Descarga su JAR desde la página de los autores de MARS, guárdalo en una ubicación estable y comprueba que abre con:

```
java -jar "/ruta/completa/a/Mars4_5.jar"
```

Sustituye esa ruta por la de tu archivo; en Windows puedes usar, por ejemplo, `C:\Herramientas\Mars4_5.jar`. No se instala dentro de Logisim ni sustituye a Questa.

Para que las herramientas del repositorio encuentren los JAR, configura estas variables con rutas completas a **archivos**, no a carpetas:

Linux, en la terminal desde la que ejecutarás las herramientas:

```
export LOGISIM_JAR="/ruta/completa/a/logisim-evolution-5.0.0-all.jar"
export MARS_JAR="/ruta/completa/a/Mars4_5.jar"
```

Windows, para una sesión de PowerShell:

```
$env:LOGISIM_JAR = "C:\Herramientas\logisim-evolution-5.0.0-all.jar"
$env:MARS_JAR = "C:\Herramientas\Mars4_5.jar"
```

Estas asignaciones solo afectan la terminal actual y los programas que se inicien desde ella. Para conservarlas en Windows, puedes añadir `LOGISIM_JAR` y `MARS_JAR` como variables de usuario, igual que se explica para la licencia en la sección 5. Los comandos de ensamblado y de evaluación siguen siendo los del README y de `docs/guide.pdf` del repositorio.

3. Abrir y comprobar la plantilla

1. Descarga el repositorio oficial del proyecto mediante **Code → Download ZIP** y extráelo completo, o utiliza la copia que te entregó el profesor. Conserva juntos `circuit/`, `docs/`, `programs/`, `images/`, `tools/` y `open.sh`. Una ruta corta y sin caracteres especiales facilita el diagnóstico, por ejemplo `C:\SMIPS` o una carpeta `SMIPS` dentro de tu usuario de Linux.
2. Abre primero Logisim Evolution **5.0.0**.
3. Selecciona **Archivo → Abrir…** y elige `circuit/smips.circ`.
4. Comprueba que no aparezcan errores de bibliotecas ni de herramientas ausentes.
5. Identifica `Board`, la placa principal; `CPU`, inicialmente vacío salvo sus diez pines; y `Memory`, la memoria ya construida. El procesador multiciclo se desarrolla dentro de `CPU` y de sus subcircuitos.
6. Conserva una copia del material original y trabaja en otra copia completa. En tu proyecto y en la entrega, mantén el circuito en `circuit/smips.circ`, que es la ruta utilizada por las herramientas.

No modifiques `Board` **ni** `Memory`**.** En `CPU` no cambies nombres, anchos ni sentidos de los pines, no los borres y recrees y no edites la apariencia del subcircuito. Puedes mover los pines dentro del lienzo de `CPU` sin alterar su apariencia. Son las condiciones R1-R2 de la orientación publicada.

La plantilla no contiene el procesador resuelto. Abrirla sin errores no significa que ya pueda ejecutar un programa S-MIPS. No uses la ausencia de salida de un programa como prueba de que la instalación falló antes de implementar la CPU.

Abrir con el lanzador del repositorio en Linux

Con el JDK instalado y `LOGISIM_JAR` configurada como se indica en la sección 2, abre una terminal en la raíz del proyecto -la carpeta que contiene `open.sh`- y ejecuta:

```
bash open.sh
```

Abre `circuit/smips.circ`. Si el circuito contiene VHDL, el lanzador también configura Questa y activa la cosimulación; antes debes completar las secciones 4-6 y definir `QUESTA_BIN` y la licencia. Sin VHDL, no necesita Questa.

Define `LOGISIM_JAR` explícitamente: el lanzador publicado tiene como ruta predeterminada una instalación de Snap que puede no existir en tu computadora o corresponder a otra versión. No copies rutas de otra computadora ni des por hecho que el lanzador localiza el instalador MSI o DEB.

En Windows puedes abrir `circuit/smips.circ` desde la interfaz gráfica y configurar Questa manualmente si lo necesitas. `open.sh` no es un lanzador nativo de Windows; la existencia de instaladores para Windows no convierte automáticamente ese script en uno compatible con PowerShell.

Cargar una imagen de programa cuando la CPU esté implementada

El repositorio ya incluye imágenes `.hex` de algunos programas en `images/`. Para cargar una de ellas, no hace falta ensamblarla nuevamente:

1. Detén los ticks automáticos del reloj.
2. Desde **la simulación de** `Board`, entra en su instancia de `Memory`: en el panel izquierdo, pestaña **Simulate/Simulación → Board → Memory**, o clic derecho sobre el bloque de memoria de la placa → **View Memory/Ver Memory**.
3. Haz clic derecho sobre la RAM `Main_Memory` → **Load Image…/Cargar imagen…** y selecciona, por ejemplo, `images/hello.hex`.
4. Vuelve a `Board`. Pulsa **Reset** con la herramienta de la mano. Si tu reset es síncrono, asegúrate de que ocurra un flanco de subida mientras el botón está pulsado; puedes avanzar el reloj manualmente durante la pulsación.
5. Libera Reset y activa **Simular → Ticks habilitados**. Con una CPU correcta, el texto aparecerá en el TTY y el LED Halt indicará la terminación.

No cargues la imagen abriendo `Memory` desde la lista de diseño: puedes estar modificando un estado aislado que no es el de la placa. El reset de la placa no borra la memoria. Para repetir una prueba que modifica datos, vuelve a cargar la imagen inicial.

No ejecutes la herramienta de regeneración de `tools/` sobre tu trabajo: reconstruye el circuito y **sobrescribe la CPU implementada**. No hace falta regenerar el circuito para abrirlo, cargar imágenes ni simularlo. El flujo oficial se describe en el README del proyecto.

Si no vas a utilizar VHDL, puedes comenzar a trabajar aquí. Mantén desactivada la simulación VHDL mientras tu diseño no la necesite.

4. Solo para VHDL: instalar Questa

Logisim no ejecuta el VHDL por sí solo: necesita Questa como simulador externo. **Importar o validar una entidad y simularla son cosas diferentes.** No basta con que el componente aparezca dibujado. Véase la ayuda oficial sobre simulación VHDL.

La combinación comprobada en Linux para este material fue **Logisim Evolution 5.0.0 + Questa Altera FPGA Starter Edition 2025.2**, distribuido con el paquete **25.1std**. Son dos numeraciones distintas: no es un error que el instalador diga 25.1 y la consola del simulador diga 2025.2.

Descargar e instalar

1. Abre la descarga oficial de Questa FPGA 25.1.
2. Descarga únicamente Questa para tu sistema. No necesitas instalar todo Quartus ni paquetes de dispositivos FPGA para simular componentes VHDL en Logisim.

3. En el asistente, selecciona **Starter Edition**, no una edición comercial que requiera otra licencia.
4. Anota la carpeta de instalación y localiza la carpeta que contiene `vsim`**,** `vcom` **y** `vlib`; en Windows sus ejecutables llevan `.exe`.

Instaladores publicados:

- Windows: `QuestaSetup-25.1std.0.1129-windows.exe`. Ejecútalo y sigue el asistente.
- Linux: `QuestaSetup-25.1std.0.1129-linux.run`. Desde su carpeta:

```
chmod +x QuestaSetup-25.1std.0.1129-linux.run
./QuestaSetup-25.1std.0.1129-linux.run
```

Son descargas grandes. Comprueba espacio disponible y los sistemas operativos admitidos por Altera para la edición elegida. La disponibilidad de un instalador Linux no significa que cualquier distribución y versión esté soportada.

Las rutas siguientes son **ejemplos**, no ubicaciones obligatorias:

- Linux: `/home/tu_usuario/altera/25.1std/questa_fse/bin`
- Windows: `C:\altera\25.1std\questa_fse\win64`

En tu instalación podría llamarse `bin`, `win64` u otra cosa. El criterio es encontrar los ejecutables, no copiar una ruta de otra computadora.

Obtener la licencia gratuita

Starter Edition es gratuita, pero requiere una licencia válida. Cada estudiante debe obtener la correspondiente a su computadora o utilizar la configuración institucional autorizada; no se reparte la licencia personal de un profesor.

1. Regístrate o inicia sesión en el Self Service Licensing Center de Altera.
2. Busca **Sign up for Evaluation or Free/No-Cost Licenses**.
3. Selecciona **Questa-Altera FPGA Starter Edition**, producto **SW-QUESTA**.
4. Registra o selecciona la computadora según los datos de hardware solicitados por el portal.
5. Genera y descarga el archivo de licencia. Guárdalo fuera del repositorio, en una ubicación estable.

El procedimiento está descrito en los recursos oficiales de Questa. Los nombres de las pantallas pueden variar. Si el registro o la descarga no te permiten continuar, comunícalo al profesor; no descargues versiones modificadas ni omitas la licencia.

5. Configurar la licencia antes de abrir Logisim

Utiliza la variable de entorno `SALT_LICENSE_SERVER`, recomendada en la configuración oficial de Questa. Aunque se llame «SERVER», también puede contener la ruta de un archivo local.

No copies literalmente las rutas de ejemplo: sustitúyelas por las tuyas. Si el laboratorio ya tiene configurada una licencia institucional, consulta antes de reemplazarla.

Linux: configuración para una sesión de terminal

Cierra previamente Logisim. En una terminal, configura la licencia y abre el JAR **desde esa misma terminal**:

```
export SALT_LICENSE_SERVER="/ruta/completa/a/tu/licencia.dat"
export QUESTA_BIN="/ruta/a/questa_fse/bin"
java -jar "/ruta/completa/a/logisim-evolution-5.0.0-all.jar"
```

El `export` solo afecta esa terminal y los programas que abras desde ella. No configura automáticamente un Logisim abierto desde el menú de aplicaciones. Repite estas órdenes en sesiones futuras o configura la variable de forma persistente en tu entorno de escritorio.

`QUESTA_BIN` permite al lanzador y a las herramientas del repositorio encontrar Questa. Si prefieres `open.sh`, configura también `LOGISIM_JAR` y, desde esa misma terminal situada en la raíz del proyecto, utiliza `bash open.sh` en lugar de la orden `java -jar`.

Windows: configuración persistente para tu usuario

1. Cierra Logisim y Questa.
2. En Inicio busca **Editar las variables de entorno de esta cuenta**. También puedes acceder desde **Configuración avanzada del sistema → Variables de entorno**.
3. En las variables de **usuario**, crea `SALT_LICENSE_SERVER` con la ruta completa de tu archivo, por ejemplo `C:\Licencias\questa-license.dat`. En este campo no añadas comillas alrededor de la ruta.
4. Acepta las ventanas y vuelve a abrir Logisim. Las terminales que ya estaban abiertas también deben cerrarse y abrirse nuevamente.

Para las herramientas del repositorio, configura también `QUESTA_BIN` con la carpeta de ejecutables de Questa, por ejemplo `C:\altera\25.1std\questa_fse\win64`, adaptada a tu instalación. Esta variable no sustituye la ruta que debes seleccionar en los ajustes de Logisim al abrirlo manualmente.

Para una prueba temporal con el JAR, en una PowerShell nueva puedes usar:

```
$env:SALT_LICENSE_SERVER = "C:\Licencias\questa-license.dat"
$env:QUESTA_BIN = "C:\ruta\a\questa_fse\win64"
java -jar "C:\Herramientas\logisim-evolution-5.0.0-all.jar"
```

La configuración temporal solo se transmite a los programas iniciados desde esa PowerShell.

Comprobar los ejecutables

Linux, sustituyendo la ruta:

```
"/ruta/a/questa_fse/bin/vcom" -version
"/ruta/a/questa_fse/bin/vsim" -version
```

Windows, en PowerShell y sustituyendo la ruta:

```
& "C:\ruta\a\questa_fse\win64\vcom.exe" -version
& "C:\ruta\a\questa_fse\win64\vsim.exe" -version
```

Mostrar una versión confirma que los ejecutables arrancan, **no que la licencia permita simular**. La comprobación completa será simular uno de tus componentes VHDL, siguiendo la sección 7.

6. Conectar Questa con Logisim

1. En Logisim, abre **Ajustes/Preferencias → Software de terceros**. Según el idioma y el menú, Ajustes puede encontrarse en **Archivo** o en **Ventana**.
2. Activa **Usar Questa Advanced Simulator para validar entidades HDL**.
3. En **Ruta de Questa Advanced Simulator**, selecciona la carpeta que contiene `vsim`, `vcom` y `vlib`, no el instalador ni el archivo de licencia.
4. Selecciona **VHDL-2002**. Es el estándar obligatorio para las entidades de este proyecto (R19), no una elección libre.

5. Deja vacías las rutas de Quartus, Vivado, ISE y OpenFPGA si no los utilizas. No hacen falta para esta integración.

Son dos configuraciones independientes: Logisim necesita la **carpeta de Questa**, y Questa necesita la **licencia disponible en el entorno**. Completar una no sustituye la otra.

El `open.sh` publicado fija VHDL-2002, pero su auxiliar desactiva la validación automática del editor al abrir el proyecto. Si utilizas el lanzador, vuelve a activar la casilla del paso 2 después de abrir. La simulación sigue necesitando compilar el código con Questa. Puedes consultar el comportamiento en `tools/OpenCircuit.java`.

7. Crear, importar y simular tus propios componentes VHDL

Qué permite la orientación

Según R18-R20 de `docs/guide.pdf`:

- Puedes sustituir por entidades VHDL **ninguno, uno o como máximo dos** de los componentes `Register_File`, `ALU` e `Instruction_Decoder`. No se permite VHDL en ningún otro componente de la CPU.
- La entidad debe llamarse como el componente que sustituye. VHDL no distingue mayúsculas de minúsculas.
- Los puertos solo pueden ser `std_logic` o `std_logic_vector`, sin `generics`.
- Solo pueden utilizarse `ieee.std_logic_1164` e `ieee.numeric_std`.
- El código debe compilar como **VHDL-2002, sin errores ni avisos**, y ser sintetizable: no se permiten `wait`, retardos `after`, archivos ni instrucciones de simulación.
- Debe conservarse la jerarquía obligatoria de R16: `Instruction_Decoder`, `Control_Unit`, `Branch_Control` y `Datapath` dentro de `CPU`; `Register_File` y `ALU` dentro de `Datapath`. `Control_Unit`, `Branch_Control`, `Datapath` y el resto del diseño se construyen con circuitos gráficos.

La posibilidad de importar código en Logisim no amplía lo permitido por el proyecto: no puedes importar una CPU completa en VHDL.

Pasos de integración

1. Trabaja en `circuit/smips.circ`. Escoge uno de los componentes permitidos, respeta su ubicación en la jerarquía y define sus puertos antes de escribir el código.
2. Utiliza **Proyecto → Añadir entidad VHDL…** para crear una entidad o **Proyecto → Importar entidad VHDL…** para cargar un archivo `.vhd` existente.
3. Abre el editor de la entidad, escribe o importa el código y utiliza **Validar y guardar**. Corrige los errores antes de continuar; no descartes cambios sin conservar una copia.
4. Selecciona la entidad del árbol del proyecto, coloca una instancia dentro de tu circuito y conecta sus puertos. Respeta los anchos: un puerto de 32 bits necesita una señal de 32 bits.
5. Si implementas `Register_File` en VHDL, recibe el reloj y el reset desde el esquema, respeta los valores de reset exigidos y utiliza el flanco de subida de `CLK`. No generes relojes internos ni derivados; R3 exige conectar directamente el reloj. La `ALU` y el `Instruction_Decoder` deben ser combinacionales.
6. Abre el circuito que contiene tu instancia VHDL y activa **Simular → Simulación VHDL activada**. Si se solicita la ruta de Questa, selecciona su carpeta de ejecutables. Espera a que termine el arranque: la consola inferior debe mostrar compilación sin errores y conexión activa; en la instalación comprobada aparecen `TCL_BINDER_CONNECTED` y `TCL_BINDER_RUNNING`.

7. Después de cambiar el código, utiliza **Simular → Reiniciar simulación VHDL** para recompilar y cargar los cambios. Reiniciar solamente el estado del circuito no sustituye este paso.
8. Guarda `circuit/smips.circ` y exporta cada entidad utilizada a su archivo `circuit/register_file.vhd`, `circuit/alu.vhd` o `circuit/instruction_decoder.vhd`, según corresponda. R23 exige que el contenido de cada archivo sea idéntico al VHDL incorporado al circuito. No se sincronizan automáticamente al editar uno de ellos.

No confundas esta integración con «Exportar HDL» o las opciones de FPGA Commander: aquí se simula una entidad VHDL como componente junto a los circuitos gráficos.

Para comprobar el funcionamiento, conecta entradas con valores conocidos y observa las salidas. En componentes secuenciales, aplica el reset y los ciclos de reloj necesarios. Prueba casos normales y casos límite, y compara las salidas con el comportamiento que definiste para el componente. Que la consola aparezca o que el código compile no demuestra por sí solo que el diseño sea correcto.

La elección de VHDL no sustituye las pruebas del proyecto ni su defensa. Los programas propios, el informe y los resultados que debes entregar son los definidos en la orientación; esta guía no añade un ejemplo HDL, un testbench obligatorio ni entregables nuevos.

8. Problemas frecuentes

- **Se abre 3.8.0 u otra versión:** el acceso directo, el comando o la asociación del `.circ` siguen apuntando a una instalación anterior. Abre explícitamente 5.0.0 y luego usa Archivo → Abrir. Como diagnóstico, utiliza el JAR con su ruta completa.
- **Biblioteca `FPArithmetic` inválida o «herramienta no encontrada»:** verifica primero la versión. No borres componentes ni aceptes una conversión y sobrescribas el original para intentar resolverlo.
- **La consola VHDL está vacía:** comprueba que el circuito que estás simulando contenga una instancia VHDL. Una placa sin esas instancias puede dejar la consola habilitada pero inactiva.
- **Se solicita Questa repetidamente:** revisa que la ruta seleccionada contenga los ejecutables, que estos arranquen y que se guardó la preferencia.
- **Error de licencia:** revisa `SALT_LICENSE_SERVER`, la existencia y vigencia del archivo y la computadora a la que se asignó. Cierra y vuelve a abrir Logisim después de configurar el entorno. No compartas el archivo al pedir ayuda.
- **Compila, pero las salidas siguen en `U`, `X` o `E`:** revisa la conexión activa, las entradas, los anchos y el reset cuando corresponda. No interpretes esos símbolos como resultados correctos. Los errores que persisten requieren diagnóstico.
- `Invalid force value: E`**:** se observó transitoriamente al arrancar una solución docente. No es una autorización para ignorar errores: si persiste, se repite al cambiar entradas o los resultados no coinciden con el comportamiento esperado, detén la comprobación y entrega el registro al profesor.
- **Los cambios de código no se reflejan:** valida y guarda, reinicia la simulación VHDL y verifica que editaste la entidad incorporada al `.circ`, no únicamente una copia externa.
- **La simulación va lenta:** espera al arranque y comienza con pruebas pequeñas. En diseños secuenciales utiliza una frecuencia de ticks moderada; un valor alto no garantiza más velocidad. La cosimulación tiene sobrecoste.
- **La sesión se desconecta:** prueba con un único proyecto con VHDL activo. Si el sistema solicita permisos de red, aplica las normas del laboratorio; no desactives el cortafuegos globalmente.
- **El programa cargado no aparece en la placa:** entra en la instancia de `Memory` desde la simulación de `Board` y vuelve a cargar el `.hex`; no lo cargues en una vista aislada del diseño.

- **El lanzador no encuentra el JAR o abre otra versión:** configura `LOGISIM_JAR` con el JAR 5.0.0. Si Java no puede ejecutar `OpenCircuit.java`, comprueba que tienes un JDK 21 o posterior disponible en esa terminal, no solo el Java incluido dentro del instalador de Logisim.
- **Questa no arranca o faltan bibliotecas del sistema:** compara tu distribución y versión con los requisitos del fabricante. No instales paquetes al azar ni cambies versiones de herramientas sin informar al profesor.

Al solicitar ayuda, adjunta sistema operativo, versión de Logisim, versión de Questa, paso exacto que falló, captura y mensajes de error. No adjuntes licencias, contraseñas ni identificadores privados de hardware.

9. Antes de comenzar el proyecto

- ☐ Logisim muestra **5.0.0** en Acerca de.
- ☐ Mi copia de la plantilla abre sin errores de bibliotecas.
- ☐ Puedo guardar y volver a abrir mi copia.
- ☐ Mi archivo de trabajo es `circuit/smips.circ`; no he modificado `Board`, `Memory` ni la interfaz o apariencia de `CPU`.
- ☐ Si uso las herramientas del repositorio, tengo configuradas las rutas `LOGISIM_JAR` y `MARS_JAR`, y el JDK está disponible en la terminal.
- ☐ Si usaré VHDL, Questa compila y simula uno de mis componentes con mi licencia y sus salidas coinciden con los resultados esperados.
- ☐ Si usaré VHDL, puedo localizar, editar, validar y volver a simular una entidad.
- ☐ Si usaré VHDL, respeto el máximo de dos componentes permitidos, VHDL-2002 y las restricciones R18-R20; los `.vhd` de `circuit/` coinciden con el código incorporado al `.circ`.
- ☐ No he incluido licencias ni instalaciones completas en el repositorio.

Alcance de la verificación: el flujo de cosimulación se comprobó en Linux con la combinación indicada. Hay descargas oficiales para Linux y Windows y se incluyen instrucciones para ambos; estas instrucciones no constituyen una certificación de que se hayan probado todas las versiones de Windows o todas las distribuciones Linux. Cada equipo debe superar las comprobaciones en su computadora antes de depender de VHDL para el proyecto.

No necesitas ninguna solución privada del docente para seguir esta guía. El único lanzador mencionado aquí es `open.sh`, incluido en el repositorio público del proyecto.

Revisión de coherencia realizada contra el README, `docs/guide.tex`, `circuit/smips.circ`, `open.sh` y sus auxiliares de la versión publicada `2666564`. Si el profesor actualiza la orientación, prevalece la versión que indique.